PROJE TASARIM RAPORU

Ders: BİL-204 \ Yazılım Mühendisliği
Bölüm/Şube: Bilgisayar Mühendisliği
Grup Adı: Grup - 4
Proje Adı: Proof-Based-Portfolio Platform
Teslim Tarihi: 04.05.2026

Grup Üyeleri:

- Ali Akbayrak (25100011409)

- Elif İpek Kaynar (24100011007)

- Mustafa Pekkirişci (25100011405)

-Rabia Şahin (25100011404)

-Selime Buse Yanal (24100011817)

-Şeyda Öksüzoğlu (25100011471)

İçindekiler

İçindekiler 2
1. Giriş 3
1.1. Sistemin Amacı 3
1.2. Tasarım Hedefleri 3
1.2.2. Güvenlik(Security) 3
1.2.3. Performans (Performance) 3
1.2.4. Bakım Yapılabilirlik (Maintainability) 4
2. Üst Düzey Yazılım Mimarisi 4
2.1. Alt Sistem Ayrıştırması 4
2.2. Donanım/Yazılım Eşlemesi 5
2.3. Kalıcı Veri Yönetimi 8
2.4. Erişim Kontrolü ve Güvenlik 13
2.4.1. Güvenlik Önlemleri ve Kararlar 13
2.4.2. Erişim Kontrol Matrisi 13
2.5. Küresel Kontrol Akışı 14
2.6. Sınır Koşulları 16
2.6.1. Başlatma 16
2.6.2. Sonlandırma 17
2.6.3. Hata Durumları 17
3. Alt Düzey Tasarım 18
3.1.Nesne Tasarım Kararları 18
3.2. Son Nesne Tasarımı 19
3.3. Paketler / Katmanlar 22
3.3.1. Dış Paketler 23
3.3.2. İç Paketler 25
3.4. Sınıf Arayüzleri 28
3.4.1. Control Sınıfları (İş Mantığı ve Oturum Yönetimi) 28
3.4.2. Entity Yapıları (Drizzle Şemaları) 34
3.4.3. Boundary Sınıfları 34
3.5. Tasarım Desenleri 38
Sözlük (Glossary) 39
Referanslar 40

1. Giriş

Bu bölümde, projenin geliştirilme amacı, çözüm sunduğu temel problemler ve sistemin inşasında temel alınan tasarım hedefleri açıklanmaktadır.

1.1. Sistemin Amacı

"Proof-Based Portfolio Platform", yazılım geliştiricilerin projelerini ve sosyal bağlantılarını modern, minimal ve doğrulanabilir bir yapıda sergileyebilmeleri için tasarlanmış full-stack bir web uygulamasıdır. Sistemin temel amacı, geliştiricilerin yetkinliklerini sadece beyan düzeyinde bırakmayıp; canlı demo bağlantıları ve kaynak kodu erişimi ile desteklenen, "kanıt temelli" (proof-based) bir dijital profil sunmalarını sağlamaktır. Bu platform, bento.me sadeliğinde bir kullanıcı deneyimi sunarak geliştiricilerin profesyonel kimliklerini hızlı ve güvenilir bir şekilde oluşturmalarına olanak tanır.

1.2. Tasarım Hedefleri

Projenin hayata geçirilmesinde rehberlik eden temel tasarım hedefleri ve kalite kriterleri aşağıda detaylandırılmıştır: **1.2.1. Kullanılabilirlik(Usability)**

Sistem, kullanıcıların herhangi bir teknik eğitim veya tasarım bilgisine ihtiyaç duymadan portföylerini yönetebilecekleri kadar sezgisel bir arayüz sunmayı hedefler. Sezgisel navigasyon ve hızlı form girişleri sayesinde kullanıcı deneyimi en üst düzeyde tutulur.

1.2.2. Güvenlik(Security)

Kullanıcı verilerinin güvenliği için iron-session ile şifrelenmiş cookie tabanlı bir oturum yapısı kullanılır. Parolalar bcrypt algoritması ile (prod ortamında cost: 12) hashlenir ve tüm API katmanlarında Zod ile veri doğrulaması yapılarak sistem bütünlüğü korunur.

1.2.3. Performans (Performance)

Genel erişime açık olan portföy sayfalarında yüksek hız sağlamak amacıyla Next.js'in SSG (Statik Site Oluşturma) ve ISR (Artımlı Statik Yeniden Oluşturma) stratejileri benimsenmiştir. Bu sayede sayfalar CDN üzerinden önbellekten sunulur ve yüklenme süreleri minimize edilir [1].

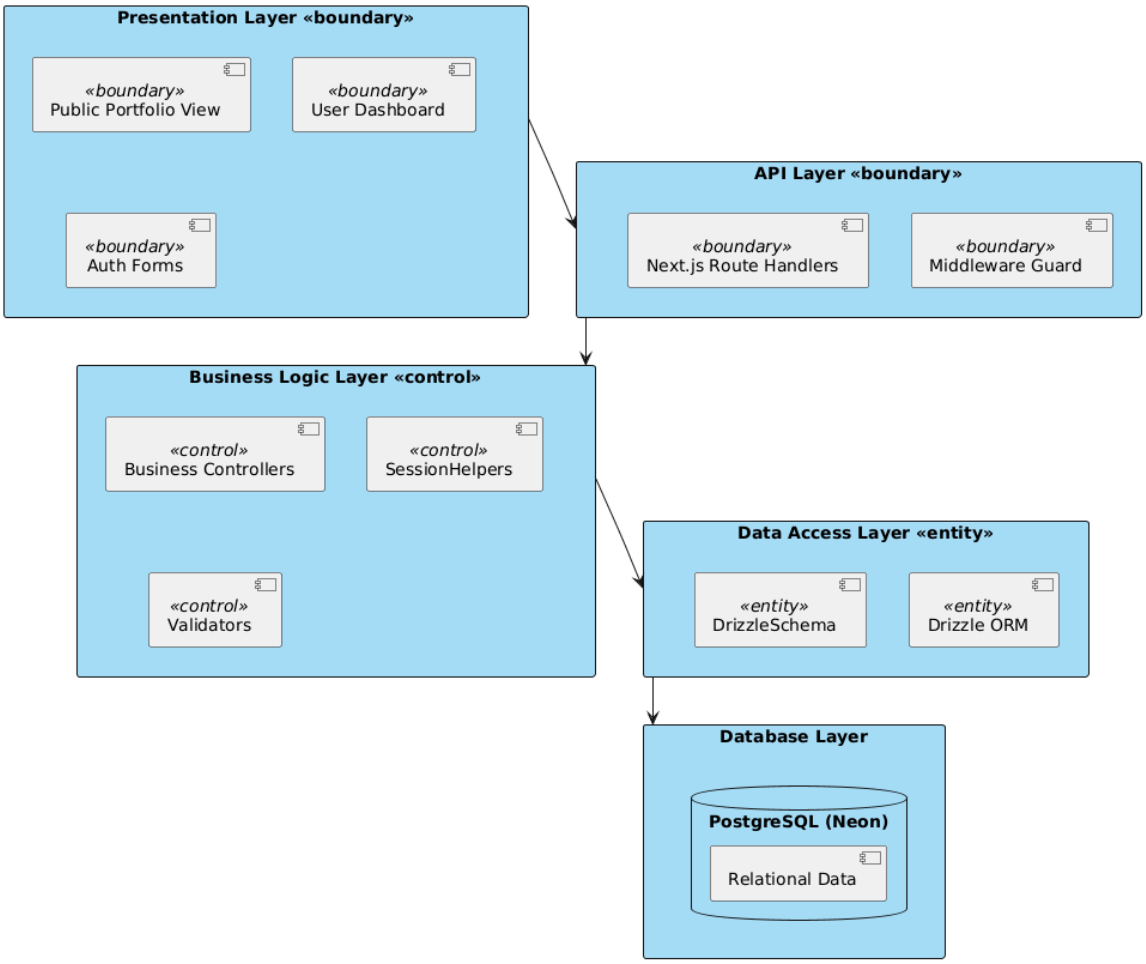
1.2.4. Bakım Yapılabilirlik (Maintainability)

Sistem mimarisi BCE (Boundary-Control-Entity) prensiplerine göre katmanlara ayrılmıştır. TypeScript strict mode kullanımı ve modüler yapı sayesinde kodun okunabilirliği artırılmış, ileride yapılacak geliştirmeler için esnek bir altyapı oluşturulmuştur.

2. Üst Düzey Yazılım Mimarisi

Bu bölümde sistemin mimari yapısı, alt sistemlere ayrıştırma, donanım/yazılım eşlemesi, veri yönetimi, erişim kontrolü, kontrol akışı ve sınır koşulları ele alınır.

2.1. Alt Sistem Ayrıştırması



Sistem beş ana katmandan oluşmaktadır. Presentation Layer, kullanıcıyla doğrudan temas kuran tüm sayfa ve form bileşenlerini barındırır. API Layer, gelen HTTP isteklerini karşılayan route handlerları ve oturum doğrulamasını yöneten Middleware Guard'ı içerir.

Business Logic Layer, iş kurallarının uygulandığı Controller sınıflarını, session yardımcılarını ve Zod doğrulama şemalarını barındırır. Data Access Layer, Drizzle ORM aracılığıyla veritabanı işlemlerini soyutlar. Database Layer ise Neon üzerinde hostlu PostgreSQL veritabanını içerir. Katmanlar arasındaki bağımlılıklar tek yönlüdür; her katman yalnızca bir alt katmanı çağırır.

2.2. Donanım/Yazılım Eşlemesi

Bu sistem, MVP yaklaşımına uygun olarak minimum donanım bağımlılığı ile, bulut tabanlı ve serverless (sunucu yönetimsiz) mimari üzerine kurulmuştur. Sistem, istemci–sunucu (client–server) modeline dayalı olarak çalışmaktadır. İstemci tarafında kullanıcılar uygulamaya herhangi bir kurulum yapmadan, modern web tarayıcıları (Chrome, Safari vb.) üzerinden erişim sağlar. Bu nedenle istemci tarafında özel bir donanım gereksinimi bulunmaz; internet bağlantısına sahip masaüstü bilgisayarlar, dizüstü bilgisayarlar, tabletler ve mobil cihazlar sistemi kullanmak için yeterlidir. Kullanıcı arayüzü React 19.x kullanılarak geliştirilmiştir. React'in Client Component yapıları tarayıcı üzerinde çalışırken, Server Component yapıları sunucu tarafında render edilerek performans ve veri erişim verimliliği artırılmaktadır.

Sunucu tarafında geleneksel fiziksel sunucu yönetimi yerine serverless mimari tercih edilmiştir. Uygulama, Vercel platformu üzerinde çalışan serverless fonksiyonlar aracılığıyla çalışır. Bu yapı sayesinde CPU, RAM ve depolama gibi donanım kaynaklarının yönetimi geliştiriciye ait değildir; sistem yüküne bağlı olarak otomatik ölçeklendirme sağlanır. Sunucu çalışma ortamı Node.js runtime üzerine kuruludur ve Next.js API route'ları bu ortamda çalışarak istemci ile iş mantığı katmanı arasında köprü görevi görür.

Sistem, full-stack yaklaşım ile Next.js 16.2.x (App Router) framework'ü kullanılarak geliştirilmiştir. Bu sayede frontend (Boundary katmanı) ve backend (API route'ları ve Control katmanı) aynı proje içerisinde konumlandırılmıştır. Programlama dili olarak TypeScript 5.x (strict mode) tercih edilmiştir. Bu tercih, tüm katmanlar arasında tip güvenliği sağlayarak hata oranını azaltmakta ve sistemin bakım yapılabilirliğini artırmaktadır.

Veri saklama ve yönetimi için PostgreSQL veritabanı kullanılmaktadır. Veritabanı, Neon platformu üzerinde serverless PostgreSQL olarak host edilmekte ve ölçeklenebilir bir yapı sunmaktadır. Uygulama ile veritabanı arasındaki iletişim Drizzle ORM aracılığıyla sağlanmaktadır. Bu yapı, veri erişim katmanında

(Entity) tip güvenli ve kontrol edilebilir sorgular yazılmasına olanak tanımakta ve BCE mimarisine uygun bir soyutlama sağlamaktadır.

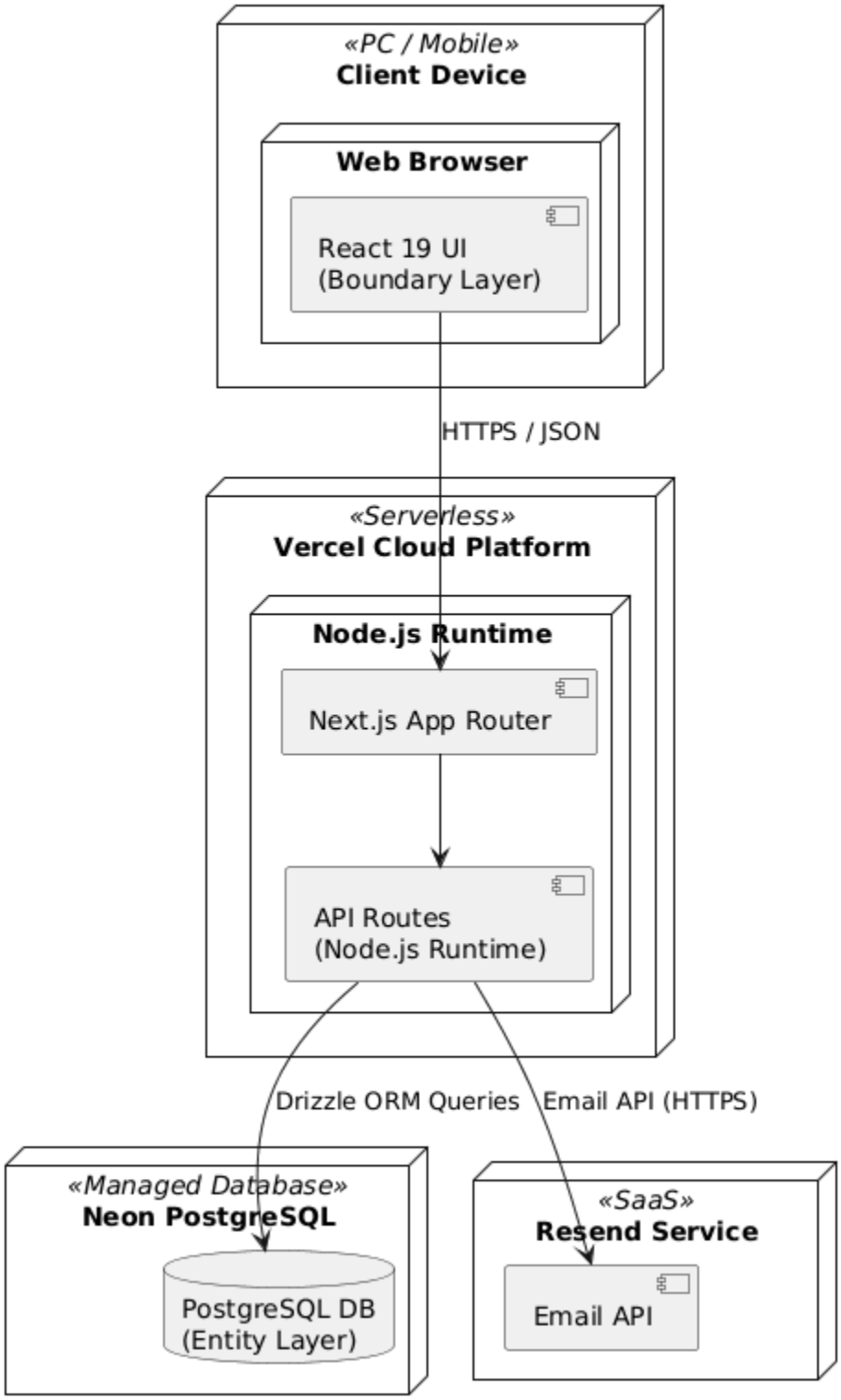
Sistemin çalıştığı işletim sistemi ortamı iki farklı bağlamda ele alınmaktadır. Geliştirme ortamında uygulama, platform bağımsız olarak Windows, macOS ve Linux işletim sistemlerinde çalışabilecek şekilde geliştirilmiştir. Production ortamında ise sistem, Vercel tarafından sağlanan Linux tabanlı serverless altyapı üzerinde çalışmaktadır. Bu yaklaşım, taşınabilirlik ve geliştirme esnekliği açısından önemli bir avantaj sağlamaktadır.

Kimlik doğrulama ve oturum yönetimi için iron-session kullanılmaktadır. Kullanıcı oturumları “portfolio-session” adlı şifrelenmiş cookie üzerinden tutulur. Bu cookie HttpOnly, production ortamında Secure ve SameSite=Lax özelliklerine sahiptir ve 7 gün süreyle geçerlidir. Kullanıcı şifreleri bcrypt algoritması ile (minimum cost değeri 10 olacak şekilde) hashlenerek saklanmakta ve hiçbir zaman düz metin olarak tutulmamaktadır.

Kullanıcıdan alınan verilerin yönetimi için React Hook Form, doğrulama işlemleri için ise Zod kütüphanesi kullanılmaktadır. Arayüz tasarımı Tailwind CSS v4 ile gerçekleştirilmiş ve shadcn/ui bileşenleri kullanılarak tutarlı ve yeniden kullanılabilir bir kullanıcı arayüzü oluşturulmuştur. E-posta işlemleri, özellikle parola sıfırlama süreçleri için Resend servisi üzerinden gerçekleştirilmektedir.

Sonuç olarak bu donanım ve yazılım eşlemesi; sistemin kurulum gerektirmeyen, ölçeklenebilir, güvenli ve bakım maliyeti düşük bir yapıda çalışmasını sağlamaktadır. Serverless mimari sayesinde donanım yönetimi geliştiriciden soyutlanmış, modern web teknolojileri kullanılarak yüksek performans ve geliştirme verimliliği elde edilmiştir. Bu yapı, projenin MVP hedefleri ile tamamen uyumlu, sürdürülebilir ve genişletilebilir bir çözüm sunmaktadır.

Deployment Diagram - Portfolio Platform (MVP)



2.3. Kalıcı Veri Yönetimi

Sistemde kalıcı olarak saklanması gereken veriler; kullanıcı kimlik bilgileri, portföy yapılandırmaları, proje içerikleri ve dış bağlantı bilgileridir. Bu verilerin bütünlüğü, tutarlılığı ve ilişkisel yapısı göz önünde bulundurularak **PostgreSQL** ilişkisel veritabanı yönetim sistemi seçilmiştir. Veritabanı, Neon platformu üzerinde serverless PostgreSQL olarak barındırılmakta; bu sayede otomatik ölçeklenme, yüksek erişilebilirlik ve geleneksel sunucu yönetimi yükü ortadan kaldırılmaktadır.

Veri erişim katmanında **Drizzle ORM** kullanılmasının temel gerekçesi; Neon'un serverless driver'ı ile doğrudan uyumlu çalışması, küçük bundle boyutu sayesinde Vercel serverless fonksiyonlarının soğuk başlangıç (cold start) sürelerini minimize etmesi ve TypeScript strict mode ile tam tip güvenliği sağlamasıdır. Drizzle, pgTable tanımları üzerinden şema yönetimini kod içinde merkezileştirir; drizzle-kit ile migration dosyaları otomatik üretilir [2].

Veri erişim stratejisi, BCE mimarisine uygun olarak tek yönlü bağımlılık üzerine kuruludur. ***lib/db/index.ts*** dosyasında singleton deseniyle yönetilen ***db*** instance'ı, uygulamanın tek veritabanı erişim noktasıdır. Controller sınıfları (Control) doğrudan bu instance üzerinden sorgularını gerçekleştirir; route handler'lar (Boundary) ise asla Drizzle instance'ına doğrudan erişemez. Geliştirme ortamında ***globalThis*** üzerinden paylaşılan singleton yapı, hot-reload sırasında bağlantı havuzunun (connection pool) tekrar tekrar oluşturulmasını engeller.

Veritabanı şeması dört ana tablodan oluşmaktadır. Aşağıdaki tabloda kolon tipleri, kısıtlamalar ve varsayılan değerler özetlenmiştir:

Tablo	Kolon	Tip	Kısıt / Varsayılan
users	***id***	***uuid***	PRIMARY KEY, ***gen_random_uuid()***
	email	***varchar(255)***	UNIQUE, NOT NULL, lowercase normalize
	passwordHash	***varchar(255)***	NOT NULL
	fullName	***varchar(100)***	—

	bio	***varchar(500)***	—
	avatarUrl	***text***	—
	createdAt	***timestamp***	DEFAULT ***now()***
portfolios	***id***	***uuid***	PRIMARY KEY, ***gen_random_uuid()***
	userId	***uuid***	FOREIGN KEY → ***users(id)***, UNIQUE, ON DELETE CASCADE
	slug	***varchar(32)***	UNIQUE, NOT NULL, regex: ***^[a-z0-9][a-z0-9-]{2,30}$***
	theme	***varchar(50)***	DEFAULT ***'minimal-light'***
	isPublished	***boolean***	DEFAULT ***false***
	createdAt	***timestamp***	DEFAULT ***now()***
	updatedAt	***timestamp***	DEFAULT ***now()***
projects	***id***	***uuid***	PRIMARY KEY, ***gen_random_uuid()***
	portfolioId	***uuid***	FOREIGN KEY → ***portfolios(id)***, ON DELETE CASCADE

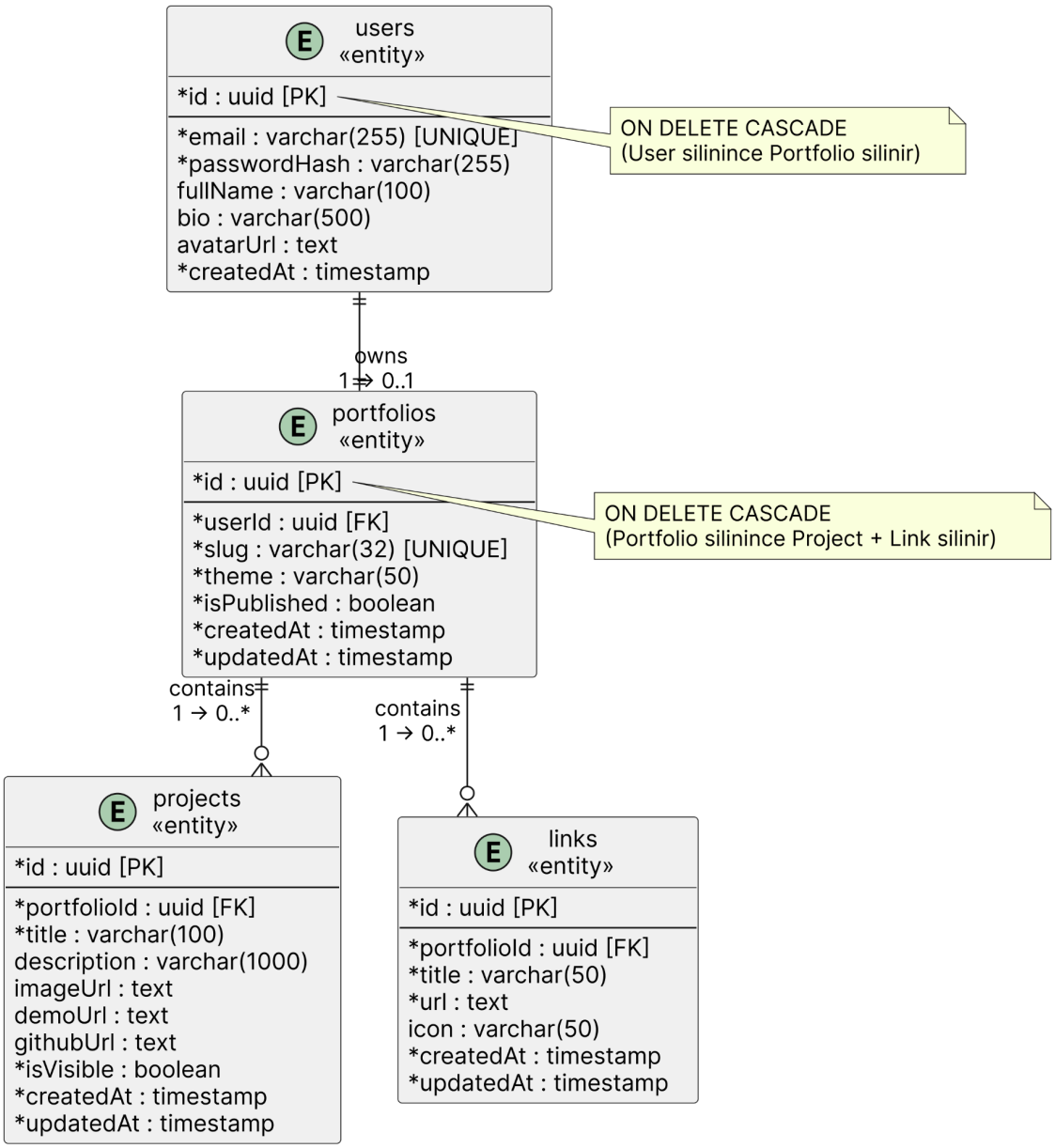
	title	***varchar(100)***	NOT NULL
	description	***varchar(1000)***	—
	imageUrl	***text***	—
	demoUrl	***text***	—
	githubUrl	***text***	—
	isVisible	***boolean***	DEFAULT ***true***
	createdAt	***timestamp***	DEFAULT ***now()***
	updatedAt	***timestamp***	DEFAULT ***now()***
links	***id***	***uuid***	PRIMARY KEY, ***gen_random_uuid()***
	portfolioId	***uuid***	FOREIGN KEY → ***portfolios(id)***, ON DELETE CASCADE
	title	***varchar(50)***	NOT NULL
	url	***text***	NOT NULL
	icon	***varchar(50)***	—
	createdAt	***timestamp***	DEFAULT ***now()***
	updatedAt	***timestamp***	DEFAULT ***now()***

İlişkiler ve Cascade Davranışı:

- User (1) — Portfolio (0..1): Bire-bir sahiplik. Kullanıcı silindiğinde bağlı portföy ON DELETE CASCADE ile otomatik silinir.
- Portfolio (1) — Project (0..*): Bire-çok içerme. Portföy silindiğinde altındaki tüm projeler kaskad olarak silinir.
- Portfolio (1) — Link (0..*): Bire-çok içerme. Portföy silindiğinde altındaki tüm bağlantılar kaskad olarak silinir.

Bu cascade yapısı, UC-12 (Hesabı ve Verileri Silme) senaryosunun tek bir veritabanı işlemiyle gerçekleşmesini sağlar; böylece veri bütünlüğü uygulama katmanına bırakılmadan veritabanı seviyesinde garanti altına alınır.

ER Diyagramı (PlantUML):


users
«entity»
*id : uuid [PK]
*email : varchar(255) [UNIQUE]
*passwordHash : varchar(255)
fullName : varchar(100)
bio : varchar(500)
avatarUrl : text
*createdAt : timestamp
ON DELETE CASCADE
(User silinince Portfolio silinir)
owns
1 → 0..1
portfolios
«entity»
*id : uuid [PK]
*userId : uuid [FK]
*slug : varchar(32) [UNIQUE]
*theme : varchar(50)
*isPublished : boolean
*createdAt : timestamp
*updatedAt : timestamp
ON DELETE CASCADE
(Portfolio silinince Project + Link silinir)
contains
1 → 0..*
contains
1 → 0..*
projects
«entity»
*id : uuid [PK]
*portfolioId : uuid [FK]
*title : varchar(100)
description : varchar(1000)
imageUrl : text
demoUrl : text
githubUrl : text
*isVisible : boolean
*createdAt : timestamp
*updatedAt : timestamp
links
«entity»
*id : uuid [PK]
*portfolioId : uuid [FK]
*title : varchar(50)
*url : text
icon : varchar(50)
*createdAt : timestamp
*updatedAt : timestamp

2.4. Erişim Kontrolü ve Güvenlik

2.4.1. Güvenlik Önlemleri ve Kararlar

- **Kimlik Doğrulama (Authentication):** ***iron-session*** kullanılarak **stateless (durumsuz)**, şifrelenmiş cookie yapısı uygulanmıştır. Sunucu tarafında herhangi bir oturum kaydı tutulmaz; oturumun sonlandırılması (logout), cookie'nin istemci tarafında temizlenmesi ve şifreli içeriğin imha edilmesiyle gerçekleşir.
- **Parola Güvenliği:** Parolalar ***bcrypt*** algoritması ile hash'lenir. Maliyet faktörü (cost factor) geliştirme ortamında **10**, üretim (production) ortamında ise kaba kuvvet saldırılarına karşı direnci artırmak amacıyla **12** olarak belirlenmiştir.
- **Girdi Denetimi:** Tüm katmanlar arası veri geçişlerinde **Zod** şemaları kullanılır. Bu sayede "Mass Assignment" ve tip tabanlı manipülasyonların önüne geçilir.
- **Hata Yönetimi:** Güvenlik hassasiyeti olan süreçlerde (örn. şifre sıfırlama) kullanıcıya "hesap bulunamadı" gibi spesifik bilgiler verilmez; sistem dışarıya idempotent ve jenerik mesajlar döner.

2.4.2. Erişim Kontrol Matrisi

İşlev / Kaynak	**Ziyaretçi**	**Kullanıcı (Genel)**	**Kullanıcı (Varlık Sahibi)**
Kayıt ve Oturum Açma	✓	✗	-
Kamu Portföyü Görüntüleme	✓	✓	✓

Kendi Portföyünü Oluşturma	✗	✓	-
Proje/Link Ekleme	✗	✗	✓
Proje/Link Düzenleme/Silme	✗	✗	✓
Portföy Yayınlama/Gizleme	✗	✗	✓

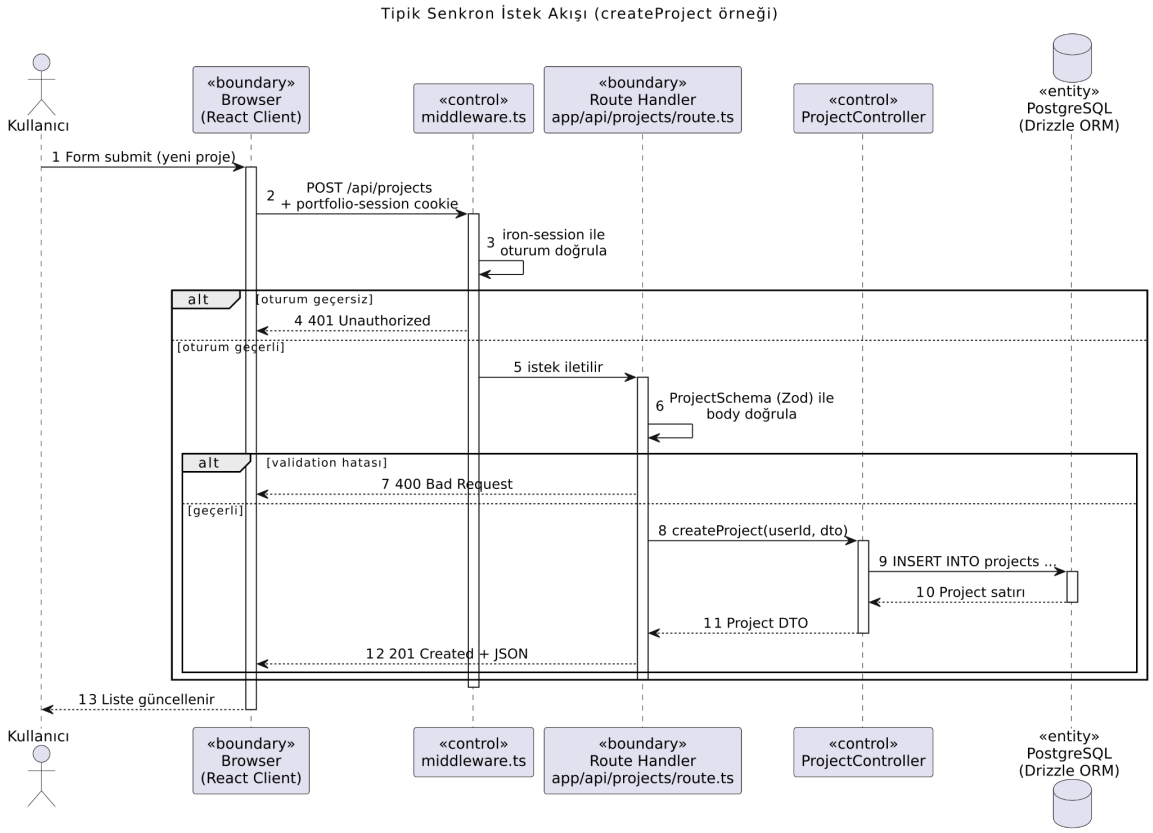
2.5. Küresel Kontrol Akışı

Sistem, senkron ve request-response tabanlı bir HTTP kontrol akışı üzerine kurulmuştur. Kullanıcıdan gelen her istek, Next.js 16 App Router üzerinde çalışan tek bir backend süreci tarafından karşılanır; mesaj kuyruğu, arka plan iş süreçleri (background job) veya websocket gibi asenkron mekanizmalar MVP kapsamında bilinçli olarak dışarıda bırakılmıştır. Bu tercih, mimari sadeliği ve Vercel sunucusuz (serverless) dağıtım modeline uyumu önceliklendirir; alternatif olarak değerlendirilen event-driven veya kuyruk tabanlı yaklaşım, ek altyapı (Redis, worker süreçleri) gerektirdiği [3] ve MVP iş akışları senkron bir modelle yeterince hızlı tamamlanabildiği için ertelenmiştir.

Tarayıcıdan gelen her istek aynı katmanlı zinciri izler. Önce ***middleware.ts*** çalışır ve ***iron-session*** ile şifrelenmiş ***portfolio-session*** çerezi üzerinden oturum doğrulaması yapılır. Ardından ***app/api/**** altındaki route handler (Boundary) HTTP detaylarını parse ederek Zod şeması ile girdiyi doğrular. Doğrulanmış veri ***lib/controllers/***

altındaki ilgili Controller sınıfına (Control) iletilir; Controller iş kurallarını uygular ve Drizzle ORM aracılığıyla PostgreSQL'e (Entity) erişir. Cevap aynı zinciri ters yönde tırmanarak istemciye döner. Bu zincir, route handler'ın asla doğrudan veritabanına dokunmaması kuralıyla korunur.

Sistemde bu temel akışın üzerine üç farklı render stratejisi oturur. Public portföy sayfaları (***app/[slug]/page.tsx***) SSG/ISR ile cache'lenir; ziyaretçi sayfa görüntülendiğinde veritabanına istek gitmez, kullanıcı bir değişiklik yayınladığında ise Controller içinden ***revalidatePath()*** çağrısı ile cache invalide edilir. Dashboard rotaları (***app/(dashboard)/****) SSR ile başlar, sonraki etkileşimler Client Component'ler içinde içinde optimistic UI ile yürür. GitHub import işlevi ise tek bir senkron HTTP çağrısı olarak modellenir. Tipik bir akışın uçtan uca gösterimi için bkz.



2.6. Sınır Koşulları

2.6.1. Başlatma

Sistem, serverless mimari üzerine kurulu olduğundan geleneksel bir sunucu başlatma süreci içermez. Geliştirme ortamında ***npm run dev*** komutu ile başlatılırken,

production ortamında Vercel'e yapılan deploy ile otomatik olarak aktif hale gelir ve Next.js 16.2.x (App Router) uygulaması Node.js runtime üzerinde gelen HTTP isteklerine göre serverless fonksiyonları tetikler.

İlk çalıştırmada, Drizzle ORM aracılığıyla Neon üzerinde barındırılan PostgreSQL veritabanına bağlantı ***lib/db/index.ts*** üzerinden singleton yapıda kurulur. MVP kapsamında başlangıç verisi bulunmaz; sistem kullanıcı etkileşimi ile veri üretir. Geliştirme ortamında isteğe bağlı olarak ***seed.ts*** ile test verisi eklenebilir.

Kimlik doğrulama sürecinde ***iron-session*** kullanılır ve her istekte ***portfolio-session*** adlı şifrelenmiş cookie ***middleware.ts*** üzerinden kontrol edilir. Geçerli oturumu olan kullanıcılar dashboard'a, olmayan kullanıcılar login/register sayfalarına yönlendirilir.

Kullanıcı arayüzü başlangıç durumu oturum bilgisine göre belirlenir. Oturumu olmayan kullanıcılar ***app/(auth)*** altındaki sayfaları görürken, oturum sahibi kullanıcılar ***app/(dashboard)*** arayüzüne erişir. Public portföy sayfaları (***app/[slug]***) ise Next.js SSG/ISR mekanizmaları ile önceden üretilir veya gerektiğinde güncellenir.

2.6.2. Sonlandırma

Sistem, serverless mimari üzerine kurulu olduğundan geleneksel bir sunucunun manuel olarak kapatılması söz konusu değildir. Vercel üzerinde çalışan serverless fonksiyonlar yalnızca istek geldiğinde çalışır ve işlem tamamlandıktan sonra otomatik olarak sonlandırılır.

Kullanıcı oturumu, logout işlemi ile sonlandırılır. Bu işlem sırasında iron-session tarafından yönetilen portfolio-session adlı şifrelenmiş cookie temizlenir ve oturum geçersiz hale getirilir. Ayrıca cookie'nin 7 günlük TTL süresi dolduğunda oturum otomatik olarak sona erer.

Veri kaybını önlemek amacıyla tüm veritabanı işlemleri PostgreSQL üzerinde ACID özelliklerine uygun şekilde gerçekleştirilir. API route'ları üzerinden yapılan işlemler doğrulanmadan ve başarıyla tamamlanmadan kalıcı hale getirilmez, böylece kesinti durumlarında veri bütünlüğü korunur.

Kullanıcı arayüzünde form verileri React Hook Form ve Zod ile doğrulanarak hatalı veri gönderimi engellenir ve kullanıcıya işlem öncesinde geri bildirim verilerek yanlışlıkla veri kaybı riski azaltılır.

2.6.3. Hata Durumları

Sistem, serverless mimari üzerinde çalıştığı için beklenmeyen hatalar (ağ kesintisi, sunucu hatası, harici servis problemleri vb.) istek bazlı ve izole şekilde ele alınır.

Vercel üzerinde çalışan serverless fonksiyonlar hata durumunda isteği sonlandırır ve istemciye uygun HTTP durum kodları (4xx, 5xx) ile geri dönüş sağlar.

API route'ları seviyesinde tüm giriş verileri Zod ile doğrulanır. Geçersiz veya eksik veri durumunda işlem gerçekleştirilmeden hata döndürülür ve hatalı verilerin veritabanına yazılması engellenerek veri bütünlüğü korunur. Route handler katmanı yalnızca HTTP seviyesinde hata yönetimi yaparken, iş mantığına ait hatalar Controller katmanında (AuthController, PortfolioController, ProjectController, LinkController) ele alınır.

Sunucu tarafında oluşan hatalar, güvenlik gereği teknik detay içermeyen jenerik mesajlarla kullanıcıya iletilir. Hata detayları yalnızca sunucu tarafında loglanır. İstemci

tarafında React Hook Form ve Zod ile anlık doğrulama yapılarak kullanıcı hataları erken aşamada tespit edilir ve kullanıcıya açık geri bildirim verilir. API hatalarında kullanıcıya uygun uyarı mesajları gösterilir.

Hata telafi kapsamında başarısız işlemler kalıcı hale getirilmez ve veritabanına yansıtılmaz. Dashboard tarafında yapılan işlemler, hata durumunda geri alınır (rollback). Oturum süresi dolmuşsa kullanıcı yeniden giriş sayfasına yönlendirilir. Harici servis hatalarında (Resend, GitHub API) işlem iptal edilir ve kullanıcıya tekrar denemesi gerektiği bildirilir.

Global hata yönetimi için Next.js App Router yapısı kullanılır. ***app/error.tsx*** ile beklenmeyen hatalar yakalanır, ***app/not-found.tsx*** ile bulunamayan sayfalar kullanıcıya uygun şekilde gösterilir.

3. Alt Düzey Tasarım

Bu bölümde sistemin sınıf düzeyindeki tasarımı, tasarım kararları, paketler, sınıf arayüzleri ve kullanılan tasarım desenleri detaylandırılır.

3.1.Nesne Tasarım Kararları

Kimlik Doğrulama Yöntemi — iron-session ve NextAuth: Başlangıçta NextAuth kullanmayı düşündük çünkü OAuth desteği hazır geliyor. Ancak MVP'de GitHub veya Google ile giriş gibi bir özelliğe ihtiyacımız yok; sadece e-posta ve parola yeterli. NextAuth'un adapter ve provider konfigürasyonu bu aşamada gereksiz karmaşıklık yaratıyordu. Bu yüzden çok daha sade olan iron-session'ı seçtik. Kararın trade-off'u ileride OAuth eklemek istersek auth katmanını yeniden yazmamız gerekebilir.

Veri Erişim Katmanı — Drizzle ve Prisma: Prisma daha yaygın ve öğrenmesi kolay bir ORM ama Neon ile birlikte kullanıldığında Prisma Accelerate gibi ek servisler gerektirebilir. Drizzle ise Neon'un serverless driverıyla doğrudan çalışıyor ve bundle boyutu çok daha küçük kalıyor. Bu da Vercel'deki başlangıç sürelerini olumlu etkiliyor.

Trade-off olarak Drizzle'ın API'si Prisma kadar sezgisel değil; bazı sorgularda SQL bilgisi gerekiyor.

Stil Çözümü — Tailwind CSS v4 ve CSS Modules: Kullandığımız shadcn/ui kütüphanesi zaten Tailwind üzerine kurulu olduğu için CSS Modules seçseydik bu bileşenleri sıfırdan uyarlamak zorunda kalırdık. Tailwind v4'ün tema sistemi de portföy temalarını yönetmeyi kolaylaştırıyor. Dezavantajı ise v4'ün henüz yeni olması; v3'e göre dokümantasyonu daha az olgunlaşmış durumda.

Public Portföy Render Stratejisi — SSG/ISR ve SSR: Portföy sayfaları çok sık değişmiyor ama çok sık ziyaret ediliyor. Bu yüzden her ziyarette veritabanına sorgu atan SSR yerine, sayfayı bir kez üretip CDN'den sunan SSG/ISR'ı tercih ettik. Kullanıcı portföyünü güncellendiğinde sadece o sayfa yeniden üretiliyor. Riski şu: revalidate doğru ayarlanmazsa ziyaretçi eski veriyi görebilir.

Proje Yapısı — Monolith ve Ayrı Frontend/Backend: Frontend ve backend'i ayrı projeler olarak geliştirmek ölçeklenebilirlik açısından daha iyi olurdu. Ama iki ayrı deploy pipeline kurmak, CORS ayarlarıyla uğraşmak ve tipler arasında senkronizasyonu sağlamak MVP için gereksiz bir yük oluşturuyordu. Next.js in App Router'ı sayesinde her şeyi tek projede yönetiyoruz. İleride kullanıcı sayısı ciddi ölçüde artarsa bu yapı yetersiz kalabilir.

3.2. Son Nesne Tasarımı

Bu bölümde, Analiz Raporu'nda sunulan sınıf diyagramı tasarım seviyesine taşınmıştır. Analiz aşamasındaki diyagram problem alanını (User, Portfolio,Project, Link) ve temel kontrol/sınır nesnelerini yüksek soyutlama düzeyinde tanımlıyordu; tasarım aşamasında bu yapı dört yönde genişletilmiştir.

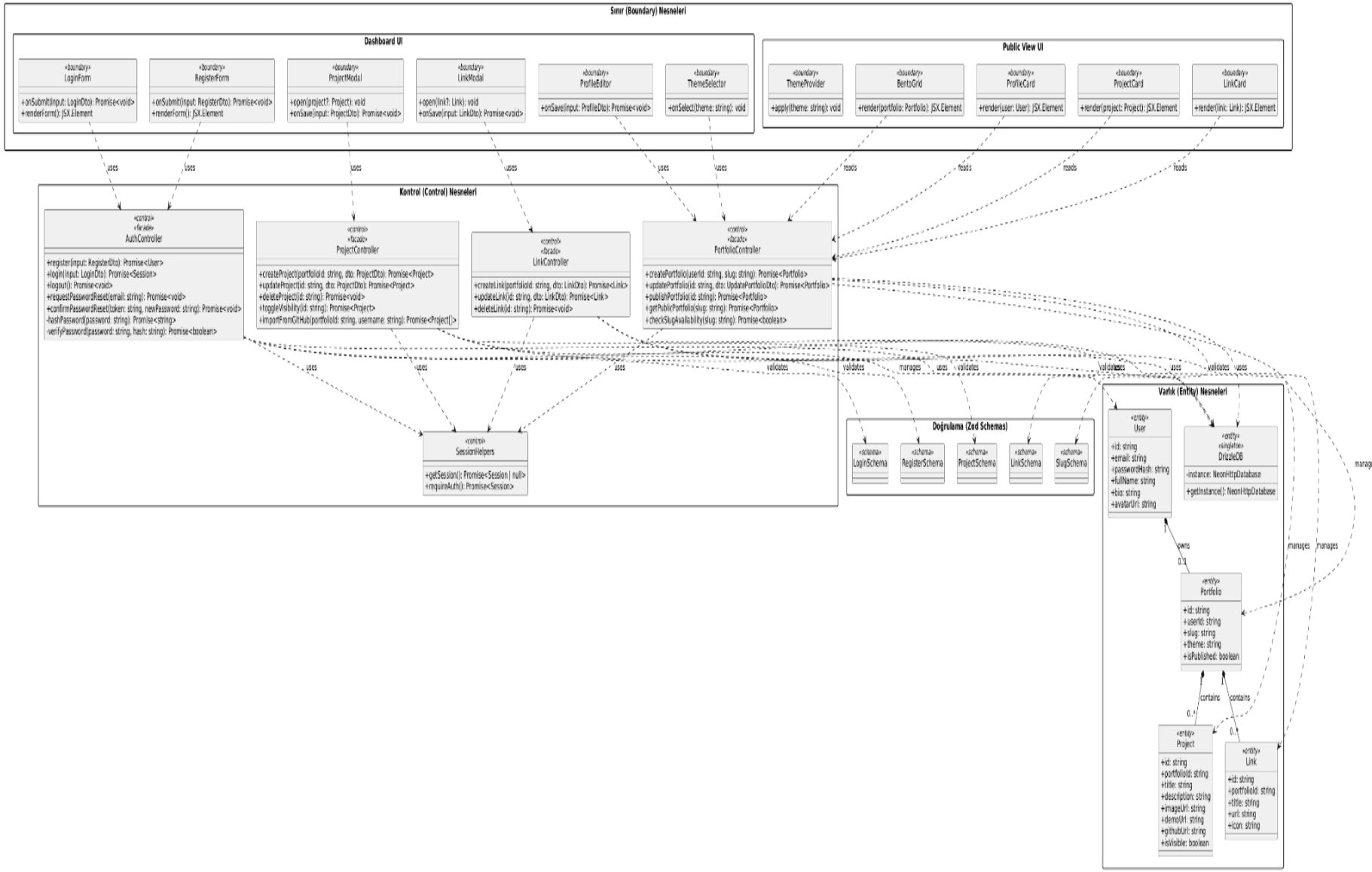
Birinci genişletme, Analiz Raporu'ndaki tek ***DashboardUI*** ve ***PortfolioViewUI*** sınıflarının gerçek React bileşenlerine ayrıştırılmasıdır. Dashboard tarafında ***LoginForm***, ***RegisterForm***, ***ProjectModal***, ***LinkModal***, ***ProfileEditor*** ve ***ThemeSelector*** bileşenleri; Public View tarafında ise ***BentoGrid***, ***ProfileCard***, ***ProjectCard***, ***LinkCard*** ve ***ThemeProvider*** bileşenleri tanımlanmıştır. İkinci genişletme, Analiz Raporu'nda yer almayan ***LinkController*** sınıfının eklenmesi ve tüm Controller sınıflarının metot listelerinin TypeScript imzalarıyla tamamlanmasıdır. Üçüncü genişletme, tasarım seviyesinde görünür hale gelen yardımcı sınıfların diyagrama dahil edilmesidir: Zod doğrulama şemaları (***RegisterSchema***, ***LoginSchema***, ***ProjectSchema***, ***LinkSchema***, ***SlugSchema***), oturum yönetimi yardımcısı ***SessionHelpers*** ve veritabanı erişim noktası olarak Singleton deseniyle tasarlanmış ***DrizzleDB*** sınıfı. Dördüncü genişletme ise ilişki türlerinin sıkılaştırılmasıdır: ***User → Portfolio***, ***Portfolio → Project*** ve ***Portfolio → Link***

ilişkileri, yaşam döngüsü bağımlılığını ifade etmek üzere composition (◆) ile gösterilmiştir; bu, veritabanı seviyesindeki ***onDelete: 'cascade'*** davranışının UML karşılığıdır.

Diyagramda BCE (Boundary–Control–Entity) ayrımı stereotipler aracılığıyla korunmuştur. ***<<boundary>>*** etiketi UI bileşenlerini, ***<<control>>*** etiketi iş mantığını taşıyan Controller sınıflarını, ***<<entity>>*** etiketi ise kalıcı veri sınıflarını işaretler. Controller sınıfları ayrıca ***<<facade>>*** stereotipi ile işaretlenmiştir; bu, route handler katmanına karmaşık doğrulama, oturum kontrolü ve veritabanı erişim adımlarını tek bir yüksek seviyeli arayüz olarak sundukları anlamına gelir. Tasarım deseni gerekçesi 3.5 Tasarım Desenleri bölümünde detaylandırılır.

Sınıflar arası bağımlılıklar üç farklı ilişki türü ile ifade edilmiştir. Boundary–Control bağlantıları dependency (**..>**) olarak gösterilmiş; Dashboard tarafındaki bileşenler Controller'ları ***uses*** etiketiyle (yazma işlemleri için), Public View bileşenleri ise ***reads*** etiketiyle (salt okuma için) çağırır. Bu ayrım, 2.4 Erişim Kontrolü bölümündeki yetki matrisinin sınıf diyagramı seviyesindeki yansımasıdır. Control–Entity bağlantıları ***manages*** etiketli dependency ile, Control–Validator bağlantıları ise ***validates*** etiketi ile gösterilmiştir.

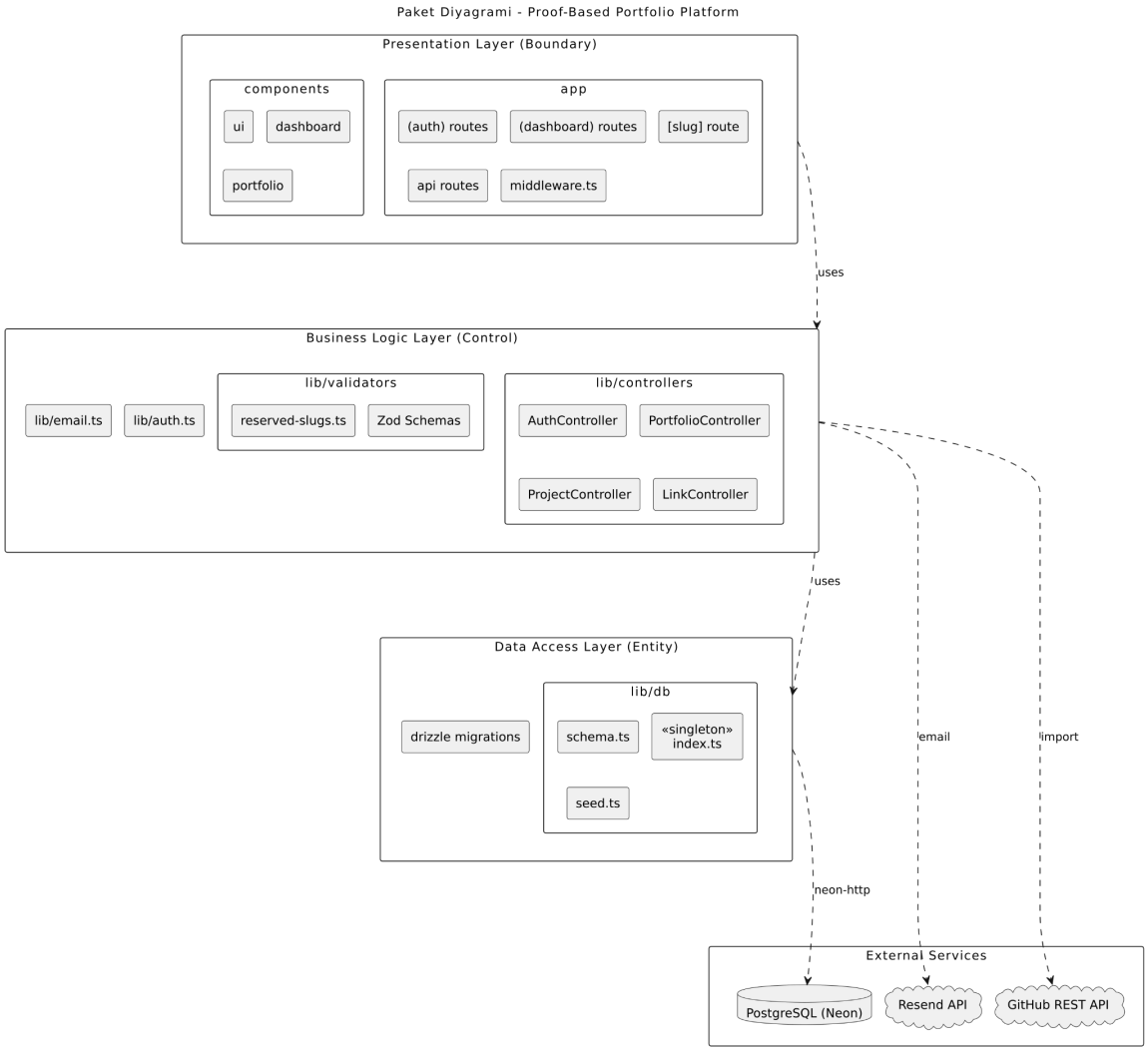
Şekil Son Nesne Tasarımı



(Yüksek çözünürlüklü fotoğrafa ulaşmak için: https://imgur.com/a/Sjj6mdG)

3.3. Paketler / Katmanlar

Sistem, BCE (Boundary-Control-Entity) prensibine uygun olarak katmanlı bir paket yapısı üzerine kuruludur. Bağımlılık yönü tek yönlüdür ve dışarıdan içeriye doğru akar: **Boundary → Control → Entity**. Yani UI bileşenleri Controller'lara çağrı yapabilir, Controller'lar Entity (Drizzle şeması) katmanına erişebilir; ancak ters yönlü bir bağımlılığa (örneğin schema dosyasının Controller'ı import etmesine) izin verilmez. Bu katmanlama hem ***lib/db/index.ts*** üzerinden tek bir veri erişim noktası sağlar hem de Controller'ların izole şekilde Vitest ile test edilebilmesini mümkün kılar.



3.3.1. Dış Paketler

Paket	Sürüm	İlgili Katman	Sistemdeki Rolü
next	16.2.x	Boundary + API	Full-stack framework; App Router üzerinden hem sayfa render'ı hem de ***app/api/**** route handler'ları aynı projede yönetir.
react	19.x	Boundary	Server Components + Client Components karışımı UI render motoru.
typescript	5.x (strict)	Tüm katmanlar	Statik tip güvenliği; ***any*** kullanımı yasaktır.
drizzle-orm + ***drizzle-kit***	son kararlı	Entity	Tip güvenli ORM; ***pgTable*** tanımları ve ***drizzle-kit*** ile migration üretimi.
@neondatabase/serverless	son kararlı	Entity	Neon üzerindeki PostgreSQL'e HTTP/serverless driver ile bağlantı.
tailwindcss	v4	Boundary	***@theme*** direktifi ile CSS değişkenleri üzerinden tema yönetimi.

Paket	Sürüm	İlgili Katman	Sistemdeki Rolü
next	16.2.x	Boundary + API	Full-stack framework; App Router üzerinden hem sayfa render'ı hem de ***app/api/**** route handler'ları aynı projede yönetir.
shadcn/ui (Radix tabanlı)	en güncel	Boundary	Erişilebilir UI primitives (Dialog, Form, Select vb.).
react-hook-form + ***zod***	son kararlı	Boundary + Control	İstemci tarafı form state yönetimi; Zod şemaları hem istemcide hem sunucu tarafında doğrulama için yeniden kullanılır.
iron-session	son kararlı	Control	***portfolio-session*** adlı şifrelenmiş cookie tabanlı oturum; ***AUTH_SECRET*** ile şifrelenir, TTL 7 gün.
bcrypt	son kararlı	Control	Parola hash'leme (cost: dev=10, prod=12).
resend	son kararlı	Control (External)	Parola sıfırlama e-postaları için SDK.
vitest	son kararlı	Test	Birim ve integration testleri.

Yukarıdaki paketlerin seçiminde değerlendirilen alternatifler ve seçim gerekçeleri (örn. iron-session vs NextAuth, Drizzle vs Prisma) Bölüm 3.1 Nesne Tasarım Kararları altında ayrıntılı olarak ele alınmıştır. Bu bölüm yalnızca seçilen paketleri ve sistemdeki rollerini tanıtmakla sınırlıdır.

3.3.2. İç Paketler

İç paketler, ekibimizin kendi yazdığı ve *portfolio-app/* projesi altında yer alan modüllerdir. Paketler BCE (Boundary–Control–Entity) prensibine göre üç ana katmana ayrılır; her paket tek bir stereotipe bağlıdır ve sorumluluk sınırı net çizilmiştir. Aşağıdaki tablo, paket yolunu, BCE stereotipini, sorumluluğunu ve içerdiği temel sınıfları/modülleri özetler.

Paket Yolu	Stereotipi	Sorumluluk	İçerdiği Temel Sınıflar / Modüller
app/(auth)/*	***<<boundary>>***	Kayıt, giriş ve parola sıfırlama sayfa rotaları. Form bileşenlerini barındırır, doğrulamayı Zod şemaları ile yapar.	***LoginForm***, ***RegisterForm***, ***ResetPasswordForm***
app/(dashboard)/*	***<<boundary>>***	Kimliği doğrulanmış kullanıcının portföy, proje ve link yönetim ekranları. Optimistic UI ile çalışır.	***ProjectModal***, ***LinkModal***, ***ProfileEditor***, ***ThemeSelector***
app/[slug]/*	***<<boundary>>***	Public portföy görüntüleme rotası; SSG/ISR ile build-time veya revalidate-on-demand olarak render edilir.	***BentoGrid***, ***ProfileCard***, ***ProjectCard***, ***LinkCard***
app/api/*	***<<boundary>>*** (API sınırı)	HTTP isteklerinin parse edildiği, Zod ile doğrulandığı ve uygun Controller'a delege edildiği route handler'ları. İş mantığı içermez.	***auth/register***, ***auth/login***, ***portfolio***, ***projects***, ***links*** route'ları

middleware.ts	***<<boundary>>***	Korumalı rotalar için session kontrolü; Chain of Responsibility deseni uygular.	***middleware()*** fonksiyonu
components/ui/*	***<<boundary>>***	shadcn/ui primitives'in proje teması ile sarılmış halleri; tüm dashboard ve public bileşenler tarafından yeniden kullanılır.	***Button***, ***Input***, ***Dialog***, ***Form***, ***Select***
components/dashboard/*	***<<boundary>>***	Dashboard'a özgü bileşik bileşenler; yalnızca giriş yapmış kullanıcılar tarafından görülür.	***ProjectModal***, ***LinkModal***, ***ProfileEditor***
components/portfolio/*	***<<boundary>>***	Public portföy görüntüleme bileşenleri; tema sistemini CSS değişkeni üzerinden uygular.	***BentoGrid***, ***ProjectCard***, ***LinkCard***, ***ThemeProvider***
lib/controllers/*	***<<control>>*** ***<<facade>>***	İş mantığının tek giriş noktaları. Route handler'ı DB ve doğrulama detaylarından yalıtır; her metot Vitest ile birim test edilebilir.	***AuthController***, ***PortfolioController***, ***ProjectController***, ***LinkController***
lib/validators/*	***<<control>>***	Zod doğrulama şemaları ve ***reserved-slugs.ts***. İstemci ve sunucuda ortak kullanılır, böylece doğrulama kuralı tek yerde tanımlanır.	***RegisterSchema***, ***LoginSchema***, ***ProjectSchema***, ***LinkSchema***, ***SlugSchema***

lib/auth.ts	***<<control>>***	iron-session sarmalı; session okuma ve "auth gerekli" kontrolü için yardımcı fonksiyonlar sunar.	***getSession()***, ***requireAuth()***
lib/email.ts	***<<control>>***	Resend SDK üzerine ince bir wrapper. ***RESEND_API_KEY*** tanımlı değilse konsola log fallback ile dev ortamında çalışmaya devam eder.	***sendPasswordResetEmail()***
lib/db/schema.ts	***<<entity>>***	Drizzle ***pgTable*** tanımları; tüm Entity tiplerinin (***$inferSelect***/***$inferInsert***) kaynağıdır.	***users***, ***portfolios***, ***projects***, ***links*** tabloları
lib/db/index.ts	***<<entity>>*** ***<<singleton>>***	***db*** instance'ının üretildiği tek nokta. Next.js dev modunda ***globalThis*** üzerinden paylaşılır, prod'da modül-seviyesi singleton olarak çalışır.	***db***
lib/db/seed.ts	***<<entity>>***	Geliştirme ortamı için örnek veri yükleyici; ***npm run db:seed*** ile tetiklenir.	***seed()***
drizzle/*	(üretilmiş)	***drizzle-kit*** tarafından şemadan otomatik üretilen migration SQL dosyaları. Elle düzenlenmez.	—

tests/unit/*	(test)	Controller'lar için Vitest birim testleri; ***db*** instance'ı mocklanarak izole çalıştırılır.	***auth-controller.test.ts***, ***portfolio-controller.test.ts***, ***project-controller.test.ts***, ***link-controller.test.ts***
tests/integration/*	(test)	API route'ları için integration testleri; gerçek bir test veritabanına karşı uçtan uca çalışır.	***register.test.ts***, ***login.test.ts***, ***publish.test.ts***, ***project-crud.test.ts***

3.4. Sınıf Arayüzleri

3.4.1. Control Sınıfları (İş Mantığı ve Oturum Yönetimi)

AuthController (***lib/controllers/auth-controller.ts***)

- ***register(data: RegisterInput): Promise<User>***: Yeni kullanıcı kaydeder ve ***User*** entity döner.
- ***login(data: LoginInput): Promise<User>***: Kimlik bilgilerini doğrular.
- ***logout(): Promise<void>: getSession()*** ile mevcut session'ı alır ve ***session.destroy()*** çağrısıyla cookie'yi temizler.
- ***requestPasswordReset(email: string): Promise<void>***: Şifre sıfırlama maili tetikler; dışarıya hata sızdırmaz.
- ***confirmPasswordReset(data: ResetInput): Promise<void>***: Token kontrolü ile şifre günceller. (Hatalar: ***InvalidToken***, ***TokenExpired***).
- ***hashPassword(plain: string): Promise<string>***: Parolayı ***bcrypt*** (cost: 10/12) ile hash'ler.
- ***verifyPassword(plain: string, hash: string): Promise<boolean>***: Hash karşılaştırması yapar.

PortfolioController (***lib/controllers/portfolio-controller.ts***)

- ***createPortfolio(userId: string, data: PortfolioInput): Promise<Portfolio>***

  Açıklama:

Kullanıcıya ait ilk portföy nesnesini oluşturur. Slug'ın veritabanında benzersiz olup olmadığını ve rezerve kelime listesine takılıp takılmadığını kontrol eder. Kullanıcı başına yalnızca bir portföy izin verilir; mevcut portföy varsa ***ConflictError*** fırlatır.

Hatalar:

- ***ValidationError*** (slug formatı geçersiz veya rezerve kelime)
- ***ConflictError*** (slug veya kullanıcı portföyü zaten mevcut)

- ***getPublicPortfolio(slug: string): Promise<PortfolioData>***

Açıklama:

Slug üzerinden kamuya açık portföy verisini getirir. Yalnızca ***isPublished=true*** olan portföyleri döner; portföye bağlı ***isVisible=true*** durumundaki Project ve Link nesnelerini birleştirerek zenginleştirilmiş bir veri yapısı sunar. Public view (SSG/ISR) tarafından kullanılır.

Hatalar:

- ***NotFoundError*** (slug bulunamadı veya portföy yayında değil)

- ***updatePortfolio(id: string, data: PortfolioInput): Promise<Portfolio>***

Açıklama:

Portföyün tema, slug veya diğer yapılandırma bilgilerini günceller. İşlem öncesinde portföyün varlığını ve istekte bulunan kullanıcının sahipliğini doğrular. Slug değişikliğinde yeni değerin çakışma kontrolü tekrar yapılır.

Hatalar:

- ***NotFoundError*** (portföy bulunamadı)
- ***AuthorizationError*** (kullanıcı portföy sahibi değil)

- ***ConflictError*** (yeni slug zaten kullanımda)

- ***publishPortfolio(id: string, status: boolean): Promise<void>***

  Açıklama:

  Portföyün ***isPublished*** durumunu belirtilen boolean değer ile günceller. Yayınlama (***status=true***) durumunda portföy public erişime açılır; gizleme durumunda ***revalidatePath()*** çağrısı ile ISR önbelleği temizlenir. Sahiplik kontrolü zorunludur.

  Hatalar:

  - ***NotFoundError*** (portföy bulunamadı)
  - ***AuthorizationError*** (yetkisiz erişim)

- ***checkSlugAvailability(slug: string): Promise<boolean>***

  Açıklama:

  Verilen slug'ın veritabanında mevcut olup olmadığını ve rezerve kelime listesine (***lib/validators/reserved-slugs.ts***) dahil olup olmadığını kontrol eder. Slug format validasyonu ***SlugSchema*** (Zod) üzerinden yapılır. Dashboard üzerindeki canlı doğrulama (debounced) için kullanılır.

  Hatalar:

  - ***ValidationError*** (slug formatı geçersiz)

ProjectController (***lib/controllers/project-controller.ts***)

- ***createProject(portfolioId: string, data: ProjectInput): Promise<Project>***

  Açıklama:

Portföy altına yeni bir proje nesnesi oluşturur. ***portfolioId***'nin istekte bulunan kullanıcıya ait olduğunu doğruladıktan sonra ***ProjectSchema*** ile gelen veriyi validasyonlar. ***isVisible*** varsayılan olarak ***true*** atanır.

Hatalar:

- ***ValidationError*** (Zod şema doğrulama hatası)
- ***NotFoundError*** (portföy bulunamadı)
- ***AuthorizationError*** (kullanıcı portföy sahibi değil)

- ***updateProject(id: string, data: ProjectInput): Promise<Project>***

Açıklama:

Mevcut projenin başlık, açıklama, URL'ler ve görsel bağlantısı gibi alanlarını günceller. Projenin varlığı ve sahiplik ilişkisi (portfolio üzerinden userId eşleşmesi) kontrol edilir.

Hatalar:

- ***NotFoundError*** (proje bulunamadı)
- ***AuthorizationError*** (kullanıcı proje sahibi değil)
- ***ValidationError*** (girdi verisi geçersiz)

- ***deleteProject(id: string): Promise<void>***

Açıklama:

Belirtilen proje nesnesini kalıcı olarak veritabanından kaldırır. İşlem öncesinde projenin varlığı ve kullanıcının yetkisi doğrulanır. Silme işlemi geri alınamaz.

Hatalar:

- ***NotFoundError*** (proje bulunamadı)
- ***AuthorizationError*** (yetkisiz silme girişimi)

- ***toggleVisibility(id: string): Promise<Project>***

  Açıklama:

  Projenin ***isVisible*** boolean değerini tersine çevirir. Bu sayede proje portföyden silinmeden geçici olarak gizlenebilir veya tekrar görünür yapılabilir. Güncellenmiş proje nesnesi döner.

  Hatalar:

  - ***NotFoundError*** (proje bulunamadı)
  - ***AuthorizationError*** (kullanıcı yetkili değil)

- ***importFromGitHub(portfolioId: string, githubUsername: string): Promise<Project[]>***

  Açıklama:

  GitHub REST API'sine sunucu tarafındaki ***GITHUB_TOKEN*** ile istek atarak kullanıcının public repolarını çeker. Dönen verileri ***Project*** nesnelerine dönüştürür; ***title=repo.name***, ***description=repo.description***, ***githubUrl=repo.html_url***. Aynı ***githubUrl'ye*** sahip proje zaten portföyde varsa güncelleme (update), yoksa yeni kayıt (create) yapar. Rate limit kontrolü dahilinde tek senkron HTTP çağrısı olarak modellenir.

  Hatalar:

  - ***NotFoundError*** (portföy bulunamadı)
  - ***AuthorizationError*** (portföy sahibi değil)
  - ***ExternalServiceError*** (GitHub API hatası veya rate limit)
  - ***ValidationError*** (dönen veri beklenen formatta değilse)

LinkController (***lib/controllers/link-controller.ts***)

- ***createLink(portfolioId: string, data: LinkInput): Promise<Link>***

  Açıklama:

  Portföy altına yeni bir bağlantı (Link) nesnesi oluşturur. URL formatı ***LinkSchema*** (Zod) ile ***https://*** prefix zorunluluğu dahilinde doğrulanır. ***icon*** alanı

boş bırakıldığında ***lib/get-icon-for-url.ts*** yardımcı fonksiyonu ile URL host'una göre otomatik ikon eşleştirmesi yapılır.

Hatalar:

- ***ValidationError*** (URL formatı geçersiz veya title uzunluğu aşıldı)
- ***NotFoundError*** (portföy bulunamadı)
- ***AuthorizationError*** (kullanıcı portföy sahibi değil)

updateLink(id: string, data: LinkInput): Promise<Link>**Açıklama:**

Mevcut bağlantının başlık, URL ve ikon bilgilerini günceller. Link'in varlığı ve portföy üzerinden sahiplik kontrolü yapılır. Güncelleme sonrası değiştirilmiş ***Link*** nesnesi döner.

Hatalar:

- ***NotFoundError*** (link bulunamadı)
- ***AuthorizationError*** (kullanıcı yetkili değil)
- ***ValidationError*** (girdi verisi geçersiz)

- ***deleteLink(id: string): Promise<void>***

  Açıklama:

  Belirtilen bağlantı nesnesini portföyden kalıcı olarak kaldırır. İşlem öncesinde link'in varlığı ve kullanıcının yetkisi ***requireAuth()*** ve portföy sahipliği karşılaştırması ile doğrulanır.

  Hatalar:

  - ***NotFoundError*** (link bulunamadı)
  - ***AuthorizationError*** (yetkisiz silme girişimi)

SessionHelpers (***lib/auth.ts***)

- ***getSession()***: ***Promise<IronSession<SessionData>>***: ***next/headers*** üzerinden şifrelenmiş session nesnesine stateless erişim sağlar.
- ***requireAuth()***: ***Promise<User>***: Oturum kontrolü yapar, yoksa ***UnauthorizedError*** fırlatır.

3.4.2. Entity Yapıları (Drizzle Şemaları)

- ***User***: Kullanıcı temel bilgileri ve kimlik doğrulama verileri.
- ***Portfolio***: Portföy ayarları ve sahiplik ilişkisi.
- ***Project***: Portföy içeriği olan doğrulanabilir projeler.
- ***Link***: Sosyal medya ve dış bağlantılar.

3.4.3. Boundary Sınıfları

3.4.3.1. Authentication Boundary

LoginForm (***components/auth/login-form.tsx***)

Açıklama: Kullanıcının email ve parola ile sisteme giriş yapmasını sağlar. Form doğrulaması React Hook Form ve Zod kullanılarak gerçekleştirilir.

Metotlar

- ***onSubmit(data: LoginFormValues): Promise<void>***
  Açıklama: Form gönderildiğinde tetiklenir. Veriler doğrulandıktan sonra ilgili Next.js API route handler'a HTTP isteği gönderilir; bu route handler, ***AuthController.login()*** metodunu çağırır.
  Hatalar:
  - ***ValidationError*** (Zod doğrulama hatası)
  - ***AuthenticationError*** (geçersiz email veya parola)

RegisterForm ***(components/auth/register-form.tsx***)

Açıklama: Yeni kullanıcı kaydı oluşturulmasını sağlar.

Metotlar

- onSubmit(data: RegisterFormValues): Promise<void>
  Açıklama: Form gönderildiğinde çalışır. Doğrulanan veriler ile API route handler'a istek gönderilir; ilgili handler AuthController.register() metodunu çağırır.
  Hatalar:
  - ValidationError
  - ConflictError (email adresi zaten kayıtlı)

3.4.3.2.Dashard Boundary

ProjectModal (***components/dashboard/project-modal.tsx***)

Açıklama: Kullanıcının proje ekleme ve düzenleme işlemlerini gerçekleştirdiğimodal bileşenidir.

Metotlar

- handleCreate(data: ProjectFormValues): Promise<void>
  Açıklama: Yeni proje oluşturmak için API route handler'a istek gönderir; ilgili handler ProjectController.createProject() metodunu çağırır.
  Hatalar:
  - ValidationError
  - AuthorizationError
- handleUpdate(id: string, data: ProjectFormValues): Promise<void>
  Açıklama: Mevcut projeyi güncellemek için API route handler'a istek gönderir; ilgili handler ProjectController.updateProject() metodunu çağırır.
  Hatalar:
  - NotFoundError
  - AuthorizationError

LinkModal (***components/dashboard/link-modal.tsx***)

Açıklama: Portföye yeni bağlantı ekleme ve düzenleme işlemlerini sağlar.

Metotlar

handleCreate(data: LinkFormValues): Promise<void>
Açıklama: Yeni bağlantı oluşturmak için API route handler'a istek gönderir; ilgili handler LinkController.createLink() metodunu çağırır.
Hatalar:

- ValidationError

- handleUpdate(id: string, data: LinkFormValues): Promise<void>
  Açıklama: Mevcut bağlantıyı güncellemek için API route handler'a istek gönderir; ilgili handler LinkController.updateLink() metodunu çağırır.
  Hatalar:
- NotFoundError

ProfileEditor (***components/dashboard/profile-editor.tsx***)

Açıklama: Kullanıcının profil bilgilerini (fullName, bio, avatarUrl) düzenlemesini sağlar.

Metotlar

- handleUpdateProfile(data: ProfileFormValues): Promise<void>
  Açıklama: Kullanıcı bilgilerini güncellemek için API route handler'a istek gönderir; ilgili handler kullanıcı güncelleme işlemini gerçekleştiren controller metodunu çağırır.
  Hatalar:
  - ValidationError
  - AuthorizationError

ThemeSelector (***components/dashboard/theme-selector.tsx***)

Açıklama: Kullanıcının portföy teması seçmesini sağlar.

Metotlar

- handleThemeChange(theme: string): Promise<void>
  Açıklama: Seçilen temayı güncellemek için API route handler'a istek gönderir; ilgili handler PortfolioController.updatePortfolio() metodunu çağırır.
  Hatalar:
  - ValidationError

3.4.3.3. Public View Boundary

BentoGrid (***components/portfolio/bento-grid.tsx***)

Açıklama: Projelerin ve bağlantıların grid düzeninde gösterilmesini sağlar.

Metotlar

- BentoGrid(props: { items: (Project | Link)[] }): JSX.Element

Açıklama: Verilen proje ve bağlantı listesini grid layout içerisinde görsel olarak render eder.

ProfileCard (***components/portfolio/profile-card.tsx***)

Açıklama: Kullanıcının profil bilgilerini görüntüler.

Metotlar

- ProfileCard(props: { user: User }): JSX.Element
  Açıklama: Kullanıcıya ait temel bilgileri arayüzde gösterir.

ProjectCard (***components/portfolio/project-card.tsx***)

Açıklama: Tek bir projenin görsel temsilini oluşturur.

Metotlar

- ProjectCard(props: { project: Project }): JSX.Element
  Açıklama: Proje bilgilerini (başlık, açıklama, bağlantılar) kullanıcıya sunar.

- **LinkCard** (***components/portfolio/link-card.tsx***)

Açıklama: Tek bir bağlantının kullanıcıya sunulmasını sağlar.

Metotlar

- LinkCard(props: { link: Link }): JSX.Element
  Açıklama: Bağlantı başlığını ve yönlendirme URL'sini arayüzde gösterir.ThemeProvider (components/portfolio/theme-provider.tsx)

Açıklama: Seçilen temayı tüm alt bileşenlere uygular. React Context kullanılarak tema bilgisi paylaşılır.

Metotlar

- applyTheme(theme: string): void

Açıklama: CSS değişkenlerini güncelleyerek seçilen temayı uygular.

3.5. Tasarım Desenleri

Proje mimarisinde, kodun sürdürülebilirliğini artırmak, bileşenler arası bağımlılığı yönetmek ve Next.js'in sunucusuz (serverless) çalışma modeline uyum sağlamak amacıyla aşağıdaki tasarım desenleri uygulanmıştır:

- **Singleton Deseni:** Drizzle veritabanı örneğinin (***db*** instance) yönetiminde kullanılmıştır (***lib/db/index.ts***). Next.js geliştirme ortamında her "hot-reload" işleminde yeni bir veritabanı bağlantısı açılmasını engellemek amacıyla, bağlantı nesnesi ***globalThis*** üzerinde tekil (singleton) olarak tutulur. Bu, veritabanı bağlantı havuzunun (connection pool) verimli kullanılmasını sağlar.
- **Facade (Görünüş) Deseni:** ***lib/controllers/*** altındaki Controller sınıflarında (AuthController, PortfolioController vb.) uygulanmıştır. Bu desen, Route Handler katmanına karmaşık veritabanı işlemlerini, Zod doğrulama süreçlerini ve iş kurallarını tek bir yüksek seviyeli arayüz üzerinden sunar. Böylece Boundary katmanı, arka plandaki karmaşık mantıktan soyutlanmış olur.
- **Chain of Responsibility (Sorumluluk Zinciri) Deseni:** Next.js ***middleware.ts*** yapısında, gelen isteklerin (request) işlenmesinde kullanılır. Bir istek sunucuya ulaşmadan önce sırasıyla oturum kontrolü (iron-session), yönlendirme kuralları ve güvenlik kontrollerinden geçer. Her halka, isteği ya bir sonraki aşamaya iletir ya da hata döndürerek süreci sonlandırır.

- **Strategy (Strateji) Deseni:** Portföy sayfalarının sunumunda ("Rendering Strategy") uygulanmıştır. Sistem, sayfa tipine göre farklı stratejiler seçer: Ziyaretçiler için "SSG/ISR" stratejisiyle yüksek performanslı statik içerik sunulurken, kullanıcı paneli için "SSR + Client Components" stratejisiyle dinamik veri yönetimi sağlanır.
- **Provider Pattern (Sağlayıcı Deseni):** React katmanında ***ThemeProvider*** ve ***SessionProvider*** yapılarında kullanılmıştır. Bu desen, tema bilgisi veya oturum durumu gibi global verilerin, bileşen ağacında "prop-drilling" (verinin en üstten en alta elle taşınması) yapılmadan her seviyeden erişilebilir olmasını sağlar.
- **Factory (Fabrika) Deseni:** Parola sıfırlama süreçlerinde "Reset Token" üretimi ve ***iron-session*** için payload oluşturma işlemlerinde kullanılmıştır. Karmaşık nesne üretim mantığı, merkezi bir noktada toplanarak kod tekrarı önlenmiştir.

Sözlük (Glossary)

Bu bölümde, "Proof-Based Portfolio Platform" projesinin tasarım ve geliştirme süreçlerinde kullanılan teknik terimler ve kavramlar tanımlanmaktadır.

- **BCE (Boundary, Control, Entity):** Sistemi kullanıcı arayüzü (Boundary), iş mantığı (Control) ve veri modelleri (Entity) olmak üzere üç ana katmana ayıran mimari desen.
- **Proof-Based (Kanıt Temelli):** Geliştiricinin yetkinliğini sadece beyan etmeyip, projenin kaynak koduna (GitHub) veya çalışan canlı demosuna verilen bağlantılarla doğrulaması yaklaşımı.
- **SSG (Static Site Generation):** Web sayfalarının talep anında değil, derleme aşamasında önceden oluşturulup sunulması yöntemi.
- **ISR (Incremental Static Regeneration):** Önceden oluşturulmuş statik sayfaların, tüm siteyi yeniden derlemeye gerek kalmadan arka planda belirli aralıklarla güncellenmesini sağlayan Next.js stratejisi.
- **ORM (Object-Relational Mapping):** Veritabanı tablolarını nesne yönelimli programlama dillerindeki sınıflara eşleyen teknoloji (Projede Drizzle kullanılmıştır).
- **Stateless (Durumsuz):** Sunucunun her istemci isteğini birbirinden bağımsız olarak ele alması, kullanıcı oturum bilgilerini sunucu belleğinde saklamaması prensibi.
- **Optimistic UI:** Kullanıcı bir işlem yaptığında (örneğin bir link eklediğinde), sunucudan onay gelmesini beklemeden arayüzün işlemin başarılı olduğu varsayımıyla anında güncellenmesi tekniği.

- **Slug:** Bir web sayfasının adresinde yer alan, kullanıcı dostu ve okunabilir benzersiz uzantı (Örnek: ***portfolio.com/hazar-urgen***).
- **Zod:** TypeScript tabanlı, çalışma zamanında veri doğrulama ve tip güvenliği sağlayan şema kütüphanesi.
- **Bcrypt:** Parolaların güvenli bir şekilde hashlenerek saklanması için kullanılan, kaba kuvvet saldırılarına karşı dirençli algoritma.

Referanslar

[1] Noves Team, "Next.js ile Modern Web Geliştirme: Kapsamlı Rehber," *Noves Digital*, Mayıs 2026. [Çevrimiçi]. Erişilebilir: [https://noves.digital/blog/nextjs-rehberi/]. [Erişim Tarihi: 4 Mayıs 2026]

[2] Drizzle Team, "Drizzle ORM - Getting Started," 2024. [Çevrimiçi]. Erişim: https://orm.drizzle.team/docs/overview. (Veri erişim katmanı tasarımı için).

[3] C. Hunt, "Event-Driven Microservices?," *DZone*, Haz. 2022. [Çevrimiçi]. Erişim: